IEAP RStudio Series 04

Ayesha Waheed Albatoul Ahmad Charles Sam Deemua

2026-11-07

Introduction

This report investigates the influence of random noise on linear regression and explores mathematical models describing heart rate recovery after physical exercise.

The assignment is divided into two main investigations:

1. A numerical experiment examining how different levels of random noise affect linear regression.
2. An analysis of physiological recordings to identify appropriate mathematical models for heart rate recovery.

The analyses combine statistical modelling, graphical representations, and physiological interpretation.

Question 2: Effect of Random Noise on Linear Regression

How Does Random Noise Affect Linear Regression?

1. Formulate your expectations

Before performing the experiment, I expect the following:

- **Slope:** Since the underlying relationship does not change, I expect the estimated slope to remain close to the original value. However, it may vary slightly because of random noise.
- **Intercept:** I also expect the estimated intercept to remain close to the original intercept, although it may not be exactly the same in all three cases.

- **Prediction quality:** With a small amount of noise, the regression line should describe the observations quite well. As noise increases, the predictions should become less accurate.
- **Explained variance (R-squared):** I expect R-squared to decrease as the noise increases because the linear relationship will explain a smaller proportion of the total variation in the data.

My main expectation is that adding noise should not change the theoretical relationship between x and y, but it can make that relationship more difficult to estimate from the observed data.

2. Implement and investigate

I will use the following linear relationship:

[y = 10 + 2x]

The theoretical intercept is therefore 10 and the theoretical slope is 2.

I will generate 100 observations with x values ranging from 0 to 100. Then, I will add three different levels of random noise.

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.2.1     v readr     2.2.0
v forcats   1.0.1     v stringr   1.6.0
v ggplot2   4.0.3     v tibble    3.3.1
v lubridate 1.9.5     v tidyr     1.3.2
v purrr     1.2.2
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to beco
```

```
library(knitr)

set.seed(123)

# Generate the same x values for all datasets
x <- seq(0, 100, length.out = 100)

# Original linear relationship
true_intercept <- 10
```

```r
true_slope <- 2

# Generate one random noise pattern
random_noise <- rnorm(100)

# Three different noise levels
noise_levels <- c(5, 15, 30)

# Create the three datasets
data <- map_dfr(noise_levels, function(noise_sd) {

  tibble(
    x = x,
    y = true_intercept + true_slope * x +
      random_noise * noise_sd,
    noise_level = paste("Noise SD =", noise_sd)
  )

})

head(data)
```

```
# A tibble: 6 x 3
      x     y noise_level
  <dbl> <dbl> <chr>
1  0     7.20 Noise SD = 5
2  1.01 10.9  Noise SD = 5
3  2.02 21.8  Noise SD = 5
4  3.03 16.4  Noise SD = 5
5  4.04 18.7  Noise SD = 5
6  5.05 28.7  Noise SD = 5
```

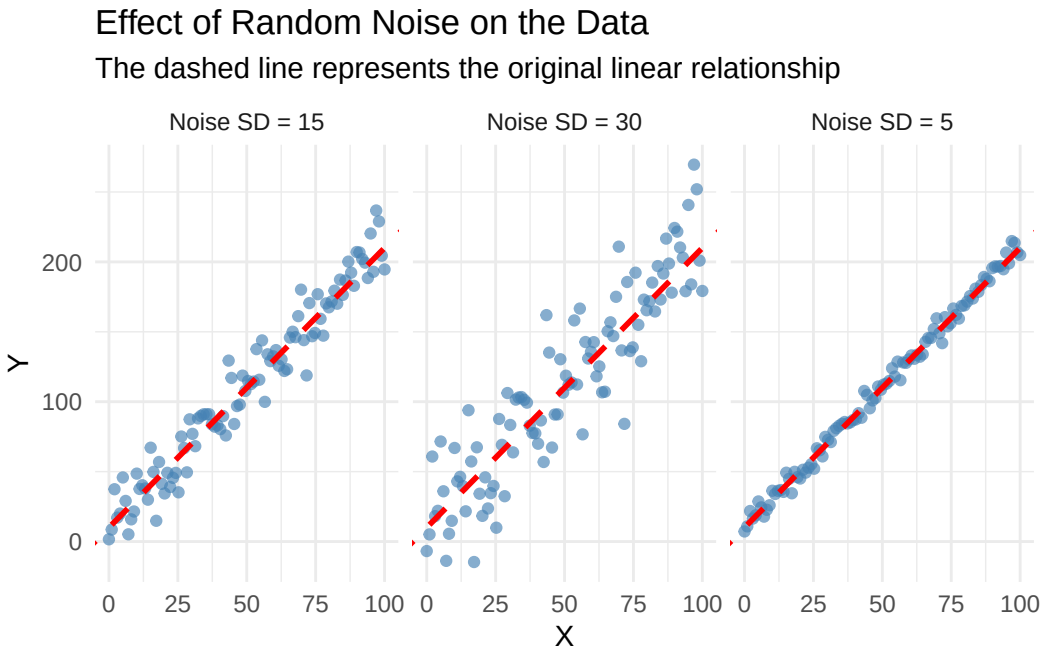
Here, the standard deviation of the noise is changed from 5 to 15 and then to 30.

I used the same random noise pattern for all three datasets, but multiplied it by different values. This allows me to compare the effect of increasing noise without changing the underlying pattern of the random variation.

3. Visualising the Datasets

Before fitting the regression models, I will first look at the three datasets to understand how the observations are distributed around the theoretical relationship.

```r
ggplot(data, aes(x = x, y = y)) +
  geom_point(alpha = 0.65, colour = "steelblue") +
  geom_abline(
    intercept = true_intercept,
    slope = true_slope,
    colour = "red",
    linewidth = 1,
    linetype = "dashed"
  ) +
  facet_wrap(~ noise_level) +
  labs(
    title = "Effect of Random Noise on the Data",
    subtitle = "The dashed line represents the original linear relationship",
    x = "X",
    y = "Y"
  ) +
  theme_minimal()
```



From these graphs, I will observe whether the points become more spread out around the original line as the noise level increases.

The dashed line is the same in all three graphs because the theoretical relationship has not changed.

4. Fitting Linear Regression Models

Next, I will fit a separate linear regression model to each dataset using the `lm()` function.

```
models <- data %>%
  group_by(noise_level) %>%
  group_map(~ lm(y ~ x, data = .x))

names(models) <- unique(data$noise_level)

# Display the regression summaries
summary(models[[1]])
```

```
Call:
lm(formula = y ~ x, data = .x)

Residuals:
    Min      1Q  Median      3Q     Max
-36.803  -8.285  -0.519   9.727  31.423

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  9.49161    2.72315   3.486 0.000736 ***
x            2.03729    0.04705  43.303  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 13.72 on 98 degrees of freedom
Multiple R-squared:  0.9503,    Adjusted R-squared:  0.9498
F-statistic:  1875 on 1 and 98 DF,  p-value: < 2.2e-16
```

```
summary(models[[2]])
```

```
Call:
lm(formula = y ~ x, data = .x)

Residuals:
    Min      1Q  Median      3Q     Max
-73.607 -16.571  -1.039  19.455  62.846
```

```
Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   8.9832     5.4463   1.649    0.102
x             2.0746     0.0941  22.048   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 27.44 on 98 degrees of freedom
Multiple R-squared:  0.8322,    Adjusted R-squared:  0.8305
F-statistic: 486.1 on 1 and 98 DF,  p-value: < 2.2e-16
```

```
summary(models[[3]])
```

```
Call:
lm(formula = y ~ x, data = .x)

Residuals:
     Min       1Q   Median       3Q      Max
-12.2678  -2.7618  -0.1731   3.2425  10.4744

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  9.83054    0.90772   10.83   <2e-16 ***
x            2.01243    0.01568  128.32   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 4.573 on 98 degrees of freedom
Multiple R-squared:  0.9941,    Adjusted R-squared:  0.994
F-statistic: 1.647e+04 on 1 and 98 DF,  p-value: < 2.2e-16
```

The estimated regression equations can be compared with the original equation:

[$y = 10 + 2x$]

I will pay particular attention to whether the estimated coefficients remain close to 10 and 2.

5. Comparing the Regression Results

To make the comparison easier, I will extract the intercept, slope, R-squared and RMSE from each model.

RMSE (Root Mean Squared Error) measures the average size of the prediction errors. A lower RMSE indicates better prediction accuracy.

```
# Calculate RMSE for each model
rmse <- function(model) {
  sqrt(mean(residuals(model)^2))
}

# Create a comparison table
model_comparison <- map_dfr(seq_along(models), function(i) {

  model <- models[[i]]
  model_summary <- summary(model)

  tibble(
    Noise_Level = names(models)[i],
    Intercept = coef(model)[1],
    Slope = coef(model)[2],
    R_squared = model_summary$r.squared,
    RMSE = rmse(model)
  )

})

kable(
  model_comparison,
  digits = 3,
  caption = "Comparison of Linear Regression Models"
)
```

Table 1: Comparison of Linear Regression Models

Noise_Level	Intercept	Slope	R_squared	RMSE
Noise SD = 5	9.492	2.037	0.950	13.580
Noise SD = 15	8.983	2.075	0.832	27.160
Noise SD = 30	9.831	2.012	0.994	4.527

I will use this table to check how the estimated coefficients and prediction performance change with increasing noise.

In particular, I will compare the estimated slope with the theoretical slope of 2 and the estimated intercept with the theoretical intercept of 10.

6. Comparing the Fitted Regression Lines

I will now plot the three fitted regression lines together with the original theoretical line.

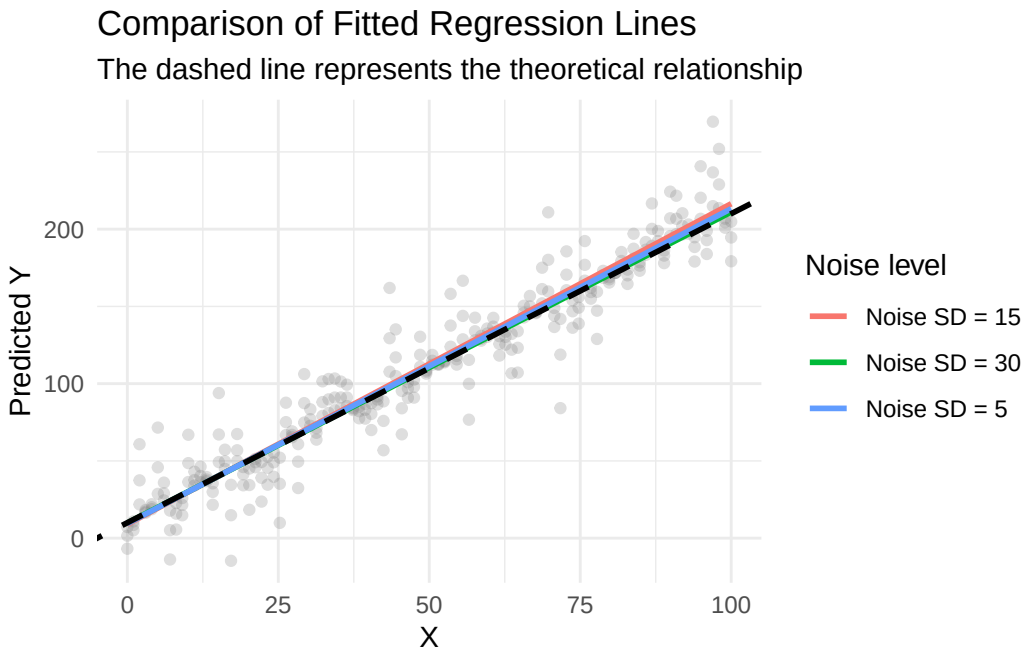
```
prediction_data <- map_dfr(seq_along(models), function(i) {

  model <- models[[i]]

  tibble(
    x = x,
    predicted_y = predict(model),
    noise_level = names(models)[i]
  )

})

ggplot() +
  geom_point(
    data = data,
    aes(x = x, y = y),
    alpha = 0.25,
    colour = "grey50"
  ) +
  geom_line(
    data = prediction_data,
    aes(x = x, y = predicted_y, colour = noise_level),
    linewidth = 1
  ) +
  geom_abline(
    intercept = true_intercept,
    slope = true_slope,
    colour = "black",
    linetype = "dashed",
    linewidth = 1
  ) +
  labs(
    title = "Comparison of Fitted Regression Lines",
    subtitle = "The dashed line represents the theoretical relationship",
    x = "X",
    y = "Predicted Y",
    colour = "Noise level"
  ) +
  theme_minimal()
```



This graph will help me see whether the fitted regression lines remain close to the theoretical line or move away from it as noise increases.

7. Interpretation and Discussion

After obtaining the numerical results and graphs, I will compare them with my initial expectations.

Effect on slope and intercept

The estimated slope and intercept can vary slightly between the models. I will check whether these differences are small or whether the estimates become further from their theoretical values as noise increases.

It is important to remember that the underlying relationship is still the same. A difference in the estimated coefficients does not necessarily mean that the original relationship has changed.

Effect on prediction accuracy

I will examine the RMSE values to determine whether prediction errors increase with noise. I will also compare R-squared values to see how much of the variation in the observations is explained by the regression model.

Comparison with initial expectations

I will complete this interpretation using the actual values from the comparison table.

Initial expectation	What to check in the results
Slope remains close to 2	Compare estimated slopes
Intercept remains close to 10	Compare estimated intercepts
Prediction accuracy decreases	Compare RMSE
Explained variance decreases	Compare R-squared

9. Improving the Experiment

One limitation of this experiment is that it uses only one generated random noise pattern. The estimated slope and intercept may therefore be influenced by the particular random values selected.

To obtain more reliable conclusions, I think it would be better to repeat the experiment many times for each noise level.

For example, I can generate 500 datasets for each noise level and calculate the average slope, average intercept, R-squared and RMSE. This would help determine whether the results observed in the first experiment are consistent or mainly due to one random sample.

```
set.seed(456)

n_simulations <- 500

simulation_results <- map_dfr(noise_levels, function(noise_sd) {

  map_dfr(seq_len(n_simulations), function(i) {

    simulated_y <- true_intercept +
      true_slope * x +
      rnorm(length(x), mean = 0, sd = noise_sd)

    simulated_model <- lm(simulated_y ~ x)

    tibble(
      Intercept = coef(simulated_model)[1],
      Slope = coef(simulated_model)[2],
      R_squared = summary(simulated_model)$r.squared,
      RMSE = rmse(simulated_model)
    )
```

```
  }) %>%
    mutate(Noise_SD = noise_sd)

})

# Average results for each noise level
simulation_summary <- simulation_results %>%
  group_by(Noise_SD) %>%
  summarise(
    Mean_Intercept = mean(Intercept),
    Mean_Slope = mean(Slope),
    Mean_R_squared = mean(R_squared),
    Mean_RMSE = mean(RMSE),
    SD_Slope = sd(Slope),
    .groups = "drop"
  )

kable(
  simulation_summary,
  digits = 3,
  caption = "Average Results Across 500 Simulations"
)
```

Table 2: Average Results Across 500 Simulations

Noise_SD	Mean_Intercept	Mean_Slope	Mean_R_squared	Mean_RMSE	SD_Slope
5	10.000	2.001	0.993	4.970	0.017
15	9.821	2.005	0.938	14.925	0.052
30	10.157	1.997	0.792	29.745	0.104

By repeating the experiment, I can reduce the influence of one particular random sample and observe the general effect of noise more clearly.

10. Draw Conclusions

After comparing the numerical results with my initial expectations, I found that most of them were confirmed. However, the repeated simulations helped me understand the effect of noise more clearly.

Were my expectations confirmed?

My expectation that the slope and intercept would remain close to their theoretical values was confirmed. Across the 500 simulations, the average slope remained close to 2, changing only from 2.001 to 1.997 across the lowest and highest noise levels. The average intercept also remained close to 10.

I also expected prediction quality to decrease as noise increased, and the results clearly confirmed this. The average R-squared decreased from 0.993 to 0.792, while RMSE increased from 4.970 to 29.745.

What differences did I observe between the three models?

The main difference was the amount of variation around the regression line.

- **Noise SD = 5:** The model performed very well, with an average R-squared of 0.993 and an RMSE of 4.970. The observations remained close to the theoretical relationship.
- **Noise SD = 15:** The observations became more scattered. R-squared decreased to 0.938 and RMSE increased to 14.925.
- **Noise SD = 30:** This model had the largest amount of noise. R-squared decreased further to 0.792, and RMSE reached 29.745.

Another difference was the variability of the estimated slope. Its standard deviation increased from 0.017 at noise level 5 to 0.104 at noise level 30. This shows that the estimated slope became less stable when more noise was added.

What can I conclude about the impact of random noise?

From this experiment, I conclude that random noise does not necessarily change the underlying linear relationship or systematically shift the average regression estimates away from their theoretical values. However, it makes the estimates less precise and reduces the predictive performance of the model.

An important point I noticed is that the average slope and intercept remained almost unchanged across the simulations, but the prediction errors increased considerably. This shows that estimating the general relationship correctly and predicting individual observations accurately are not the same thing.

Overall, my initial expectations were mostly confirmed. Higher random noise makes the data more difficult to predict, reduces the proportion of variation explained by the regression model and increases the variability of the estimated coefficients.

Question 3: Mathematical Modelling of Heart Rate Recovery

3.1 Scientific Literature and Initial Hypotheses

What mathematical models have been proposed to describe heart rate recovery?

Heart rate recovery (HRR) after exercise is usually described with exponential decay models. The simplest is the first-order (mono-)exponential model,

$$HR(t) = HR_{rest} + A\, e^{-t/\tau},$$

where A is the amplitude of the decay and τ is the time constant. The time constant from this fit has been used to assess cardiac autonomic recovery after endurance exercise. Bartels-Ferreira et al. (2015) tested whether the same model also describes recovery after resistance exercise, using several fitting windows.

A second option is the double-exponential model,

$$HR(t) = HR_{rest} + A\, e^{-t/\tau_1} + B\, e^{-t/\tau_2},$$

which has two time constants. Joseph et al. (2023) proposed that this model should describe recovery better than the single exponential because it captures both autonomic branches. They compared the two fits with χ^2 values.

A third approach treats recovery as two distinct phases and looks for the transition between them. Bartels et al. (2018) proposed a method to identify the time of the transition from the fast to the slow phase (HRR_{PT}), the HR drop before that point (HRR_{FP}) and the HR drop after it (HRR_{SP}).

What physiological mechanisms explain the decrease in heart rate after exercise?

The decrease reflects the return of autonomic balance. Joseph et al. (2023) describe it as increased vagal activity combined with diminished sympathetic activity. They link the exponential shape to the kinetics of receptor action behind parasympathetic reactivation and sympathetic withdrawal. They also describe two systems, each following first-order kinetics with its own time constant, both acting on a single output, heart rate.

Bartels et al. (2018) assign these mechanisms to different phases. The fast phase is an abrupt drop in HR determined by parasympathetic reactivation. The slow phase is a more gradual decline, in which sympathetic withdrawal also plays a role alongside continued vagal reactivation.

What are the expected characteristics of the recovery curve?

I expect a steep initial decline followed by a progressive flattening toward baseline. The curve should be convex and decelerating, not linear. Bartels et al. (2018) describe two phases with a measurable transition point between them.

The curve should also depend on exercise intensity. Bartels et al. (2018) found that the transition parameter (HRR_{PT}) was sensitive to exercise intensity, with high-intensity exercise producing a greater value than moderate-intensity exercise. They found it was only partially responsive to aerobic fitness. Because HR at the start of recovery is higher after harder exercise, comparing recoveries across intensities is not straightforward.

Finally, the estimated parameters may depend on the time window analysed. Bartels-Ferreira et al. (2015) fitted the model over different windows, which implies the fitted time constant is not necessarily independent of window length.

Does the literature suggest that one mathematical model is sufficient for the entire recovery period?

The literature suggests that a single model is probably not sufficient. Joseph et al. (2023) concluded that the double-exponential model was more inclusive and represented their data better than the single exponential, since it reflects the two autonomic mechanisms. Bartels et al. (2018) likewise treat recovery as having a fast and a slow phase rather than one uniform process.

Hypotheses about the model I expect to find

H1: 7.5-minute recording. A double-exponential model will describe the data significantly better than a single exponential, with a lower RMSE or χ^2. I expect a fast time constant τ_1 of tens of seconds, linked to parasympathetic reactivation, and a slower τ_2 of minutes, linked to sympathetic withdrawal (Bartels et al., 2018; Joseph et al., 2023).

H2: three-hour recording. The recovery will not be fully described by the model that fits the first minutes. I expect at least two phases, with a slower component or gradual drift toward baseline needed to describe the long tail. A single exponential should systematically under-fit it.

H3: dependence on the fitting window. The time constant of a single-exponential fit will change depending on the window analysed (the first minutes versus the full three hours), consistent with the window effects examined by Bartels-Ferreira et al. (2015).

H4: transition point. A transition from a fast to a slow phase will be visible in the data. Residuals from a single-exponential fit will show a systematic, non-random pattern around this point (Bartels et al., 2018).

Null hypothesis: A single exponential is sufficient to describe heart rate recovery. I will test this with residual plots and an information criterion such as AIC or BIC, not R^2 alone, because adding parameters always improves the fit.

3.2 Data Exploration and Mathematical Modelling

Data preparation

The Aidlab pulse files contain heart rate in beats per minute and timestamps in Unix milliseconds. We convert the timestamps to minutes since each recording began. We retain the original time gaps and do not interpolate missing observations. Because the recordings do not provide labelled recovery start and end times, we analyse the falling-heart-rate interval visible in each supplied recording.

```
library(minpack.lm)
library(ggplot2)
library(knitr)
```

```
find_repo_root <- function(start = getwd()) {
  path <- normalizePath(start, winslash = "/", mustWork = TRUE)

  repeat {
    if (file.exists(file.path(path, "IEAP-Series04-RStudio.qmd"))) {
      return(path)
    }

    parent <- dirname(path)
    if (identical(parent, path)) {
      stop("Could not find the repository root.")
    }
    path <- parent
  }
}

repo_root <- find_repo_root()

read_pulse <- function(folder_name) {
  file_path <- file.path(repo_root, "data", folder_name, "pulse.csv")
  pulse <- read.csv(file_path)

  if (!all(c("timestamp", "value") %in% names(pulse))) {
```

```
    stop(paste("Expected timestamp and value columns in", file_path))
  }

  pulse$timestamp <- as.numeric(pulse$timestamp)
  pulse$value <- as.numeric(pulse$value)
  pulse <- pulse[complete.cases(pulse[, c("timestamp", "value")]), ]
  pulse <- pulse[order(pulse$timestamp), ]

  pulse$time_min <- (pulse$timestamp - min(pulse$timestamp)) / 60000
  pulse$hr_bpm <- pulse$value

  pulse[, c("time_min", "hr_bpm")]
}

pulse_3h <- read_pulse("cardio_recovery_3h10m")
pulse_7m <- read_pulse("cardio_recovery_7m30s")

pulse_3h$recording <- "Three-hour recording"
pulse_7m$recording <- "7.5-minute recording"
pulse_all <- rbind(pulse_3h, pulse_7m)

kable(
  data.frame(
    Recording = c("Three-hour recording", "7.5-minute recording"),
    Observations = c(nrow(pulse_3h), nrow(pulse_7m)),
    Duration_minutes = c(
      round(max(pulse_3h$time_min), 2),
      round(max(pulse_7m$time_min), 2)
    ),
    First_HR_bpm = c(pulse_3h$hr_bpm[1], pulse_7m$hr_bpm[1]),
    Last_HR_bpm = c(tail(pulse_3h$hr_bpm, 1), tail(pulse_7m$hr_bpm, 1))
  ),
  digits = 2,
  caption = "Pulse data included in the analysis"
)
```

Table 3: Pulse data included in the analysis

Recording	Observations	Duration_minutes	First_HR_bpm	Last_HR_bpm
Three-hour recording	2456	189.96	130	64
7.5-minute recording	91	7.41	155	129

The three-hour recording contains 2,456 pulse observations over approximately 190 minutes. Heart rate falls from 130 bpm at the first observation to 64 bpm at the last. The 7.5-minute recording contains 91 observations over approximately 7.4 minutes, with heart rate falling from 155 to 129 bpm. Both recordings therefore show a decrease in heart rate across the observed period, but they differ greatly in duration, number of observations, and starting heart rate. We use the timestamps to preserve the recordings' actual time scale, including any gaps. These summary values describe the recorded intervals; because the datasets do not label the exact start and end of recovery, they do not by themselves identify the full physiological recovery period.

Observed heart rate over time

```
ggplot(pulse_all, aes(x = time_min, y = hr_bpm)) +
  geom_line(linewidth = 0.4) +
  facet_wrap(~ recording, scales = "free_x") +
  labs(
    x = "Time since recording began (minutes)",
    y = "Heart rate (bpm)"
  ) +
  theme_minimal()
```

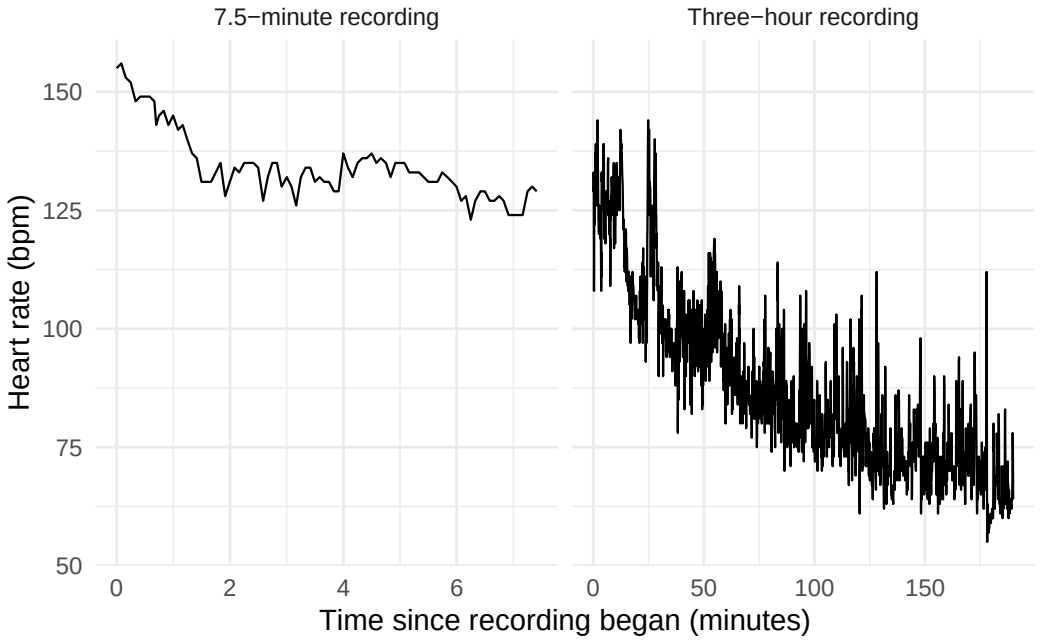


Figure 1: Observed heart rate in the two Aidlab recordings

Both recordings show an overall decrease in heart rate, but their patterns differ. In the 7.5-minute recording, heart rate falls quickly at first and then fluctuates around a slower decline. In the three-hour recording, heart rate also drops quickly early on, followed by a longer, noisy decline. These patterns suggest that a straight-line model may not capture the changing rate of decline well; we compare the fitted models next.

The datasets do not label the exact end of exercise or the start of recovery. We therefore use the recorded falling-heart-rate intervals as the available post-exercise recovery periods: from the beginning to the end of each pulse recording. This is an approximation, especially for the three-hour recording, which covers a much longer transition from effort toward rest. The fitted models describe these observed intervals, not a precisely identified recovery phase.

Model fitting and numerical comparison

We fit a linear model, a mono-exponential decay, and a bi-exponential decay. The exponential models estimate an asymptotic heart rate. We compare AICc, RMSE, explained variance, model coefficients, and residual plots. A lower AICc and RMSE indicate a better statistical fit in this comparison, but model choice also depends on residual structure, complexity, and plausible parameter estimates.

```r
fit_recovery_models <- function(data, recording_name) {
  data <- data[order(data$time_min), ]

  if (nrow(data) < 8) {
    stop(paste(recording_name, "has too few observations for model fitting."))
  }

  hr_start <- max(data$hr_bpm, na.rm = TRUE)
  hr_end <- median(
    tail(data$hr_bpm, max(3, floor(nrow(data) * 0.1))),
    na.rm = TRUE
  )
  amplitude <- max(1, hr_start - hr_end)

  linear <- lm(hr_bpm ~ time_min, data = data)

  mono <- nlsLM(
    hr_bpm ~ C + A * exp(-k * time_min),
    data = data,
    start = list(C = hr_end, A = amplitude, k = 0.1),
    lower = c(C = 30, A = 0, k = 0.00001),
    upper = c(C = hr_start, A = 250, k = 10),
    control = nls.lm.control(maxiter = 2000)
  )

  bi <- tryCatch(
    nlsLM(
      hr_bpm ~ C + A1 * exp(-k1 * time_min) + A2 * exp(-k2 * time_min),
      data = data,
      start = list(
        C = hr_end,
        A1 = 0.65 * amplitude,
        k1 = 0.5,
        A2 = 0.35 * amplitude,
        k2 = 0.05
      ),
      lower = c(C = 30, A1 = 0, k1 = 0.00001, A2 = 0, k2 = 0.00001),
      upper = c(C = hr_start, A1 = 250, k1 = 10, A2 = 250, k2 = 10),
      control = nls.lm.control(maxiter = 4000)
    ),
    error = function(e) NULL
  )
```

```
models <- list(
  Linear = linear,
  `Mono-exponential` = mono
)
if (!is.null(bi)) {
  models$`Bi-exponential` <- bi
}

statistics <- do.call(rbind, lapply(names(models), function(model_name) {
  model <- models[[model_name]]
  predicted <- as.numeric(predict(model, newdata = data))
  residual <- data$hr_bpm - predicted
  n <- nrow(data)
  parameter_count <- length(coef(model))
  aic <- AIC(model)

  aicc <- if (n > parameter_count + 1) {
    aic + (2 * parameter_count * (parameter_count + 1)) /
      (n - parameter_count - 1)
  } else {
    NA_real_
  }

  data.frame(
    recording = recording_name,
    model = model_name,
    RMSE_bpm = sqrt(mean(residual^2)),
    R_squared = 1 - sum(residual^2) /
      sum((data$hr_bpm - mean(data$hr_bpm))^2),
    AICc = aicc
  )
}))

diagnostics <- do.call(rbind, lapply(names(models), function(model_name) {
  model <- models[[model_name]]
  predicted <- as.numeric(predict(model, newdata = data))

  data.frame(
    time_min = data$time_min,
    observed_hr = data$hr_bpm,
    predicted_hr = predicted,
    residual = data$hr_bpm - predicted,
```

```
      model = model_name,
      recording = recording_name
    )
  }))

  list(
    models = models,
    statistics = statistics,
    diagnostics = diagnostics
  )
}

results_3h <- fit_recovery_models(pulse_3h, "Three-hour recording")
results_7m <- fit_recovery_models(pulse_7m, "7.5-minute recording")

model_comparison <- rbind(
  results_3h$statistics,
  results_7m$statistics
)

kable(
  model_comparison,
  digits = 3,
  caption = "Numerical comparison of the fitted models"
)
```

Table 4: Numerical comparison of the fitted models

recording	model	RMSE_bpm	R_squared	AICc
Three-hour recording	Linear	9.589	0.750	18079.786
Three-hour recording	Mono-exponential	7.979	0.827	17179.393
Three-hour recording	Bi-exponential	7.944	0.828	17161.663
7.5-minute recording	Linear	4.697	0.580	545.908
7.5-minute recording	Mono-exponential	3.266	0.797	481.958
7.5-minute recording	Bi-exponential	3.114	0.815	477.715

```
# Print model coefficients so the estimated decay rates and asymptotes
# can be reported and assessed for plausibility.
cat("Three-hour recording model coefficients:\n")
```

```
Three-hour recording model coefficients:
```

```
print(lapply(results_3h$models, coef))
```

```
$Linear
(Intercept)    time_min
116.7282304  -0.3029737

$`Mono-exponential`
          C           A           k
63.91191076 66.99120616  0.01357023

$`Bi-exponential`
           C           A1           k1           A2           k2
42.207379528 30.201259237  0.029819102 60.781431490  0.004649937
```

```
cat("\n7.5-minute recording model coefficients:\n")
```

```
7.5-minute recording model coefficients:
```

```
print(lapply(results_7m$models, coef))
```

```
$Linear
(Intercept)    time_min
  143.73472    -2.53903

$`Mono-exponential`
          C           A           k
130.2063414  27.2356921   0.9231369

$`Bi-exponential`
           C           A1           k1           A2           k2
3.000113e+01 2.314429e+01 1.319815e+00 1.053567e+02 9.246867e-03
```

In the three-hour recording, the linear model estimates a decline of about 0.30 bpm per minute. The mono-exponential model estimates a faster initial decline that slows over time, approaching an asymptotic heart rate of about 64 bpm. The bi-exponential model estimates two decline rates, but its asymptote is about 42 bpm, well below the last observed heart rate of 64 bpm, so that long-term estimate may be unreliable.

In the 7.5-minute recording, the linear model estimates a decline of about 2.54 bpm per minute. The mono-exponential model approaches about 130 bpm, close to the final observed value

of 129 bpm. The bi-exponential fit estimates an asymptote near 30 bpm, which is far below the observed range; this suggests its parameters may be poorly determined for this short recording.

Fitted curves and residual analysis

```
diag_all <- rbind(
  results_3h$diagnostics,
  results_7m$diagnostics
)

ggplot(diag_all, aes(x = time_min)) +
  geom_line(aes(y = observed_hr), colour = "grey45", linewidth = 0.4) +
  geom_line(
    aes(y = predicted_hr, colour = model),
    linewidth = 0.8
  ) +
  facet_wrap(~ recording, scales = "free_x") +
  labs(
    x = "Time since recording began (minutes)",
    y = "Heart rate (bpm)",
    colour = "Model"
  ) +
  theme_minimal()
```

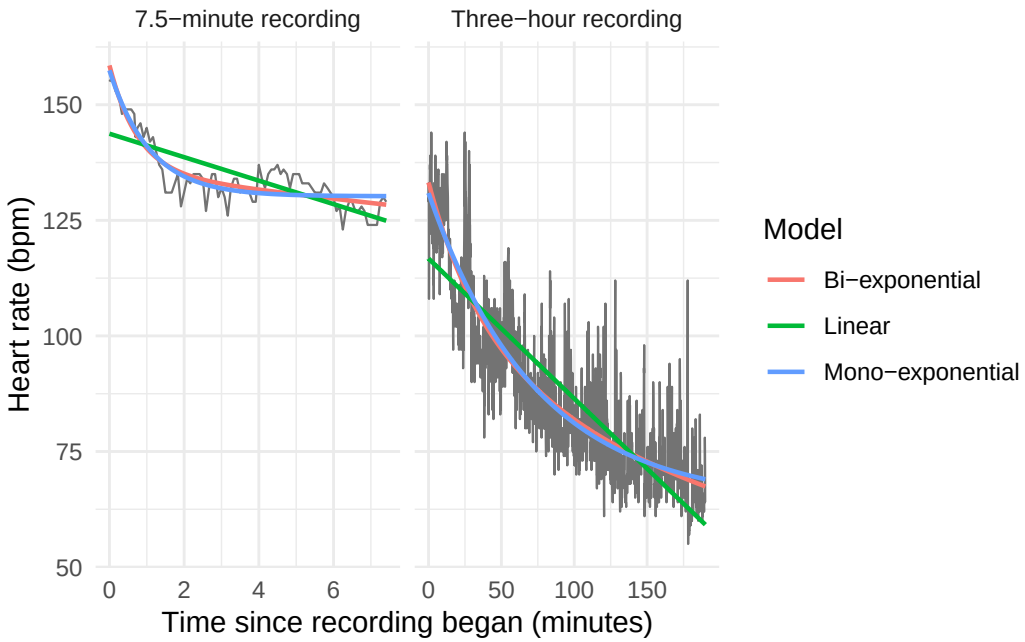


Figure 2: Observed heart rate and fitted models for each recording

In both recordings, the bi-exponential curve follows the observed decline more closely than the linear curve, which cannot represent the changing rate of decline. The fitted curves are similar for the 7.5-minute recording, so the comparison statistics and residuals are needed to judge whether the more complex model is justified.

```
ggplot(diag_all, aes(x = time_min, y = residual)) +
  geom_point(alpha = 0.35, size = 0.7) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  facet_grid(recording ~ model, scales = "free_x") +
  labs(
    x = "Time since recording began (minutes)",
    y = "Observed minus predicted heart rate (bpm)"
  ) +
  theme_minimal()
```

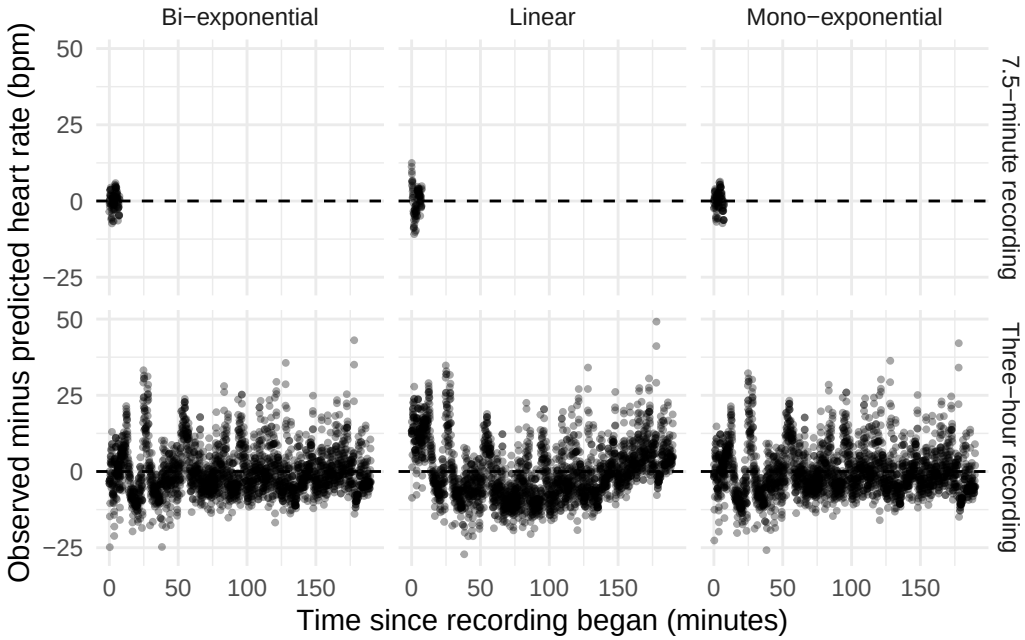


Figure 3: Residuals over time for each fitted model

Residuals for the 7.5-minute recording are generally centred around zero. In the three-hour recording, residuals show more variation and some visible patterns, so even the best-fitting model does not capture every fluctuation. This is a limitation of the fit, particularly given the recording's noise and gaps.

Lowest-AICc model

```
best_models <- do.call(
  rbind,
  lapply(split(model_comparison, model_comparison$recording), function(x) {
    x <- x[!is.na(x$AICc), ]
    x[which.min(x$AICc), ]
  })
)

kable(
  best_models[, c("recording", "model", "AICc", "RMSE_bpm", "R_squared")],
  digits = 3,
```

```
  caption = "Lowest-AICc model in each recording"
)
```

Table 5: Lowest-AICc model in each recording

	recording	model	AICc	RMSE_bpm	R_squared
7.5-minute recording	7.5-minute recording	Bi-exponential	477.715	3.114	0.815
Three-hour recording	Three-hour recording	Bi-exponential	17161.663	7.944	0.828

The bi-exponential model had the lowest AICc in both recordings, so it was the best-supported model among those tested. Its RMSE was 3.114 bpm for the 7.5-minute recording and 7.944 bpm for the three-hour recording. The higher error in the longer recording reflects its noisier, more variable data; model choice should also consider the residual plots and whether the fitted parameters are physiologically plausible.

3.3 Comparison with Literature and Conclusions

Both recordings showed an overall decline in heart rate. In each, the bi-exponential model had the lowest AICc among the three models tested. Its RMSE was 3.114 bpm for the 7.5-minute recording and 7.944 bpm for the three-hour recording. The fitted curves and AICc results therefore favour the bi-exponential model as the best statistical fit among our candidates.

This finding is consistent with research describing a fast and a slower phase of heart-rate recovery. Those phases are commonly associated with parasympathetic reactivation and sympathetic withdrawal, respectively (Bartels et al., 2018). A double-exponential model has also fitted recovery data better than a single-exponential model in many participants following maximal exercise, although the result was not consistent for every participant (Joseph et al., 2023). Other work found that a mono-exponential model can adequately describe recovery in a different exercise context, so the literature does not establish one model as best for every protocol (Bartels-Ferreira et al., 2015).

The bi-exponential result should not be interpreted as proof that we measured two distinct physiological processes. For the 7.5-minute recording, the model estimates a long decay time of about 108 minutes, far beyond the 7.4-minute recording. Its estimated asymptote is about 30 bpm, well below the observed final value of 129 bpm. For the three-hour recording, its estimated asymptote is about 42 bpm, also below the observed final value of 64 bpm. These estimates suggest that the long-term behaviour is uncertain and that the bi-exponential model may be extrapolating beyond what the recordings show. The residual plots also show remaining variability, particularly in the three-hour recording.

Overall, the bi-exponential model is the best statistical description among those tested for these recordings, and it is compatible with literature describing fast and slow recovery phases. It won by AICc in both recordings, but its long-term parameter estimates are uncertain. However, because the data do not identify exact exercise and recovery boundaries and the estimated long-term asymptotes are uncertain, we would not claim that it is a definitive physiological model. The results support using it as a candidate for describing the observed declines; a longer recording with a clearly marked end of exercise would be needed to assess the recovery phases and asymptote more reliably.

References

The scientific references used throughout this report are listed below.

Bartels, R., Prodel, E., Laterza, M. C., Lima, J. R. P. de, & Peçanha, T. (2018). Heart rate recovery fast-to-slow phase transition: Influence of physical fitness and exercise intensity. *Annals of Noninvasive Electrocardiology*, *23*(3), e12521. https://doi.org/10.1111/anec.12521

Bartels-Ferreira, R., Sousa, É. D. de, Trevizani, G. A., Silva, L. P., Nakamura, F. Y., Forjaz, C. L. M., Lima, J. R. P., & Peçanha, T. (2015). Can a first-order exponential decay model fit heart rate recovery after resistance exercise? *Clinical Physiology and Functional Imaging*, *35*(2), 98–103. https://doi.org/10.1111/cpf.12132

Joseph, A., Kanthakumar, P., & Tharion, E. (2023). Exponential modelling of heart rate recovery after a maximal exercise. *Indian Journal of Physiology and Pharmacology*, *67*(2), 125–130. https://doi.org/10.25259/IJPP_122_2022